Lean Proof Strategy Proposal for the Scheduling Rules Document

Stuart Swan, Platform Architect, Siemens EDA

17 May 2026

0. Goal and scope

The goal is to formalize the scheduling rules document's **mathematical proof core** in Lean: trace compatibility, witness construction, cutpoint correspondence, elaboration, and one-way liveness/deadlock preservation. The Lean proof would certify the formalized theorem stack under the stated assumptions, not every informal prose statement in the document.

The scheduling rules document's own framing is conditional: the formal results depend on assumptions, observation-boundary choices, witness-selection conditions, and liveness/progress premises.

The Lean development should therefore be organized around explicit theorem instances and assumption bundles, rather than hidden global axioms.

The primary scheduling rules document on which this doc is based is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/catapult_user_view_scheduling_rules.pdf

The latest version of this Lean proof strategy document is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/Lean_proof_strategy.pdf

The Matchlib examples kit that contains training slides, examples, etc., is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/tree/master

---

1. Central comparison object

The proof should be parameterized by a selected comparison instance:

```
structure ComparisonInstance where
  Sys_ref    : System
  RTL_B      : System
```

```
Sigma_ref   : Alphabet
Sigma_post  : Alphabet
Sigma_DUT   : Alphabet

env       : ReactiveEnvironment Sigma_post
refChoice   : SysRefChoice

combNetwork? : Option CombNetwork

assumptions : AssumptionBundle
witness     : WitnessBundle
contracts   : ModelingContracts
```

Sys_ref is the chosen source reference model for the theorem instance:

- ordinary bounded-FIFO case: Sys_ref = Sys_B;
- elaborated case: Sys_ref = Sys_B^elab(E) for a well-formed elaboration package E.

This follows the updated Appendix Q direction: the formal theorems do not require a unique algorithm that constructs Sys_B^elab; instead, the chosen Sys_ref is part of the theorem instance.

---

2. Foundation layer: bucketed executions, not flat traces

The primary execution model should be **cycle-bucketed**, not a flat sequence of eps | tick | sigma e. The reason is that the document allows multiple same-cycle Σ-events without imposing artificial order among independent same-cycle events. The earlier review correctly noted that a flat trace would either impose a fake linearization or require an added bucket abstraction anyway.

Use:

```
structure State where
  cycle  : Nat
  epsPos : Nat
  store  : Store
```

with explicit advancement rules:

-- ε-steps advance epsPos only.
-- Same-cycle Σ-events advance epsPos only.
-- Clock ticks advance cycle and reset epsPos.

Define buckets:

```
structure CycleBucket (Σ : Type) where
  cycle       : Nat
  startState  : State
  epsPrefix   : List EpsStep
  sigmaEvents : Finset Σ
  sigmaOrder  : PartialOrderOn sigmaEvents
  epsSuffix   : List EpsStep
  endState    : State
  endQuiescent : Prop

def ExecutionBucketed (Σ : Type) :=
  List (CycleBucket Σ)

def InfiniteExecutionBucketed (Σ : Type) :=
  Nat → CycleBucket Σ
```

This provides the right foundation for:
- ε-quiescent cutpoints;
- rendezvous same-cycle commits;
- R2C combinational fixed-point buckets;
- J.1 ideal induction over unordered or partially ordered same-cycle units;
- B2/B3 temporal reasoning over infinite bucketed executions.

Important separation: ExecutionBucketed is the semantic carrier for state advancement, ε-quiescence, occupancy updates, R2C fixed points, and fairness/progress reasoning. It is not itself the cross-model trace-compatibility predicate.

In particular, system-level compatibility $\tau_{ref,system} \approx_{sys} \tau_{post,system}$ must not be defined as equality of CycleBucket lists, equality of bucket cycle numbers, or bucket-by-bucket equality of sigmaEvents. The document's Appendix J compatibility relation is an ordered cross-model observable-compatibility predicate. It preserves the explicitly required happens-before / $\prec_J$ constraints, atomic multi-endpoint units, E1–E5 obligations, and matched value labels, while allowing independent observable units to commute or to

be grouped into different cycle buckets in Sys_ref and RTL_B.

Thus:
• bucket equality may be required inside explicitly atomic units, such as rendezvous Push/Pop pairs and SyncChannel/ac_sync handshakes;
• bucket decomposition may be used to prove local state-transition and enablement facts;
• but CompatibleSys / ≈_sys is defined over observable units and their mandated partial order, not over identical bucket decompositions.

---

3. Events, operation instances, and source order

Define dynamic operation universes:

```
structure DynMsgInstance where
  proc      : Process
  channel   : Channel
  kind      : MsgKind     -- Push, Pop, PushNB, PopNB
  blocking  : Bool
  sourceLoc : SourceLoc
  instanceId : Nat

structure FrontierItem where
  proc      : Process
  itemKind  : FrontierItemKind
  sourceLoc : SourceLoc
  instanceId : Nat
```

Core source-order objects are witness-specific. They range only over dynamic operation instances that actually occur in the compared execution / witness. Untaken branches and unexecuted loop iterations are not members of this dynamic universe.

```
Qexec : Process → Set DynMsgInstance
QOmega : Process → Set DynMsgInstance
Omega : Process → Set FrontierItem
```

Qexec,P is the execution-level dynamic-instance universe. It includes executed failed non-blocking polls that may produce no Σ-event.

Omega,P is the frontier/order universe used by Front, NextFront, WFG, E3/E5, BoundaryReq attribution, and source-order reasoning. Failed NB-only executed polls are not included in Omega,P merely because they executed.

QOmega,P denotes the message-operation-instance slice of Omega,P:
QOmega,P := Qexec,P ∩ Omega,P

```
def MsgInstanceOfFrontierItem
(C : ComparisonInstance)
(P : Process)
(x : FrontierItem) : Option DynMsgInstance :=
...

structure DynamicUniverse
(C : ComparisonInstance)
(P : Process)
(W : WitnessBundle C) where
Q_exec : Set DynMsgInstance
Ω : Set FrontierItem
Q_Ω : Set DynMsgInstance

qomega_characterization :
∀ q,
q ∈ Q_Ω ↔
q ∈ Q_exec ∧
∃ x,
x ∈ Ω ∧
MsgInstanceOfFrontierItem C P x = some q

message_frontier_items_have_exec_instance :
∀ x q,
x ∈ Ω →
MsgInstanceOfFrontierItem C P x = some q →
q ∈ Q_exec ∧ q ∈ Q_Ω

qomega_items_have_frontier_item :
∀ q,
q ∈ Q_Ω →
```

∃ x,
x ∈ Ω ∧
MsgInstanceOfFrontierItem C P x = some q

executed_only :
∀ x,
x ∈ Ω →
ExecutedInWitness C W P x

leP :
{x : FrontierItem // x ∈ Ω} →
{y : FrontierItem // y ∈ Ω} →
Prop

leP_partial_order :
PartialOrder leP

prefS :
SyncCall →
Set {x : FrontierItem // x ∈ Ω}

suffS :
SyncCall →
Set {x : FrontierItem // x ∈ Ω}

MsgInstanceOfFrontierItem is defined only for frontier items that correspond to message-operation dynamic instances. Non-message frontier items, such as pure synchronization or signal-anchor items, map to none. The field Q_Ω is therefore not a second independent universe: it is exactly the set of executed dynamic message-operation instances whose corresponding message frontier item appears in Ω. This keeps Q_exec, Ω, and Q_Ω type-distinct while making their relationship explicit for BoundaryReq, Front, NextFront, WFG, and E3/E5 issue-order reasoning.

Thus source order ≤_P is not a static total order over syntactic program statements. It is a partial order over the witness-bundle-specific dynamic frontier-item universe Ω. The selector component W.selector determines ε-modeled source choices, while W.nbMap records the dynamic-instance correspondence for failed non-blocking polls that produce no Σ-event.

Non-blocking calls need explicit modeling:

- failed PushNB / PopNB polls are executed dynamic instances in Qexec, but produce no Σ-event;
- successful non-blocking calls produce the corresponding committed Push_c(v) / Pop_c(v) event;
- repeated NB calls are distinct dynamic source instances even if a request signal remains physically asserted.

This matches the document's issue/commit treatment of NB calls.

---

4. Reactive environment and modeling contracts

4.1 Reactive environment

"Fixed stimulus" should be modeled as a causal global strategy over Σ-history, not merely as a fixed stream of per-process inputs.

```
structure ReactiveEnvironment (Σ : Type) where
  inputAt : SigmaHistory Σ → ExternalInputs
  causal :
    ∀ h₁ h₂,
      SamePrefix h₁ h₂ →
      inputAt h₁ = inputAt h₂
```

This directly addresses the earlier issue that reactive stimulus must be interpreted as the same global Σ-causal behavior, not a function of local process history.

4.2 Modeling contracts

Direct-input contracts and array-access contracts should be parallel top-level modeling contracts, not partly folded into RRules and partly left separate.

```
structure ModelingContracts (C : ComparisonInstance) where
  directInputs : DirectInputContract C
  arrays       : ArrayAccessMappingRules C
```

This keeps RRules close to the document's actual R-rule set and treats direct-input and external-array obligations as environment/modeling contracts.

---

5. Direct-input contract and late sampling

The direct-input contract should contain only primitive environmental stability/update obligations. The fact that late sampling preserves the logical anchor should be a **derived theorem**, not a separate hypothesis.

The document says hls_direct_input permits late physical sampling because the environment holds the signal stable after reset, while hls_direct_input_sync permits controlled updates only at the designated synchronization handshake. The document also states that logical signal Read/Write events are timestamped at the E2 anchor, not at any later physical RTL sampling cycle; late sampling is ε-internal.

Define:

The direct-input stability rule applies only within a single reset epoch. The scheduling document explicitly permits dynamic resetting during normal HW operation, including resetting of direct-input-controlled regions.

```
def NoRelevantResetBetween
(C : ComparisonInstance)
(sig : Signal)
(t₁ t₂ : Cycle) : Prop :=
¬ ∃ r,
t₁ < r ∧
r ≤ t₂ ∧
ReceiverResetAffectsSignal C sig r

structure DirectInputContract (C : ComparisonInstance) where
stableWithinResetEpoch :
∀ sig t₁ t₂,
DirectInput sig →
AfterAllReceiversOutOfReset C sig t₁ →
t₁ ≤ t₂ →
NoRelevantResetBetween C sig t₁ t₂ →
InputValue C.env sig t₁ = InputValue C.env sig t₂
syncControlledUpdateOnlyAtSync :
∀ sig s t,
DirectInputSyncControlled sig s →
ValueChangesAt C.env sig t →
SyncHandshakeOccursAt C s t
```

Then prove:

```
theorem lateSamplingPreservesLogicalAnchor
 (C : ComparisonInstance)
 (hDI : DirectInputContract C)
 (sig : Signal)
 (anchor sampleT : Cycle)
 (hWindow : DirectInputAnchorWindow C sig anchor sampleT)
 (hNoReset :
   NoRelevantResetBetween C sig anchor sampleT) :
 InputValue C.env sig anchor =
 InputValue C.env sig sampleT
```

Bucket-location rule:
The logical Read(sig,val) / Write(sig,val) event belongs only to the E2 anchor bucket β_anchor. A physical late sample, if modeled, is an ε-step in a later bucket. The late ε-step creates no new Σ-event. lateSamplingPreservesLogicalAnchor is used only to justify the value recorded in the logical anchor bucket.

```
structure DirectInputBucketWF (C : ComparisonInstance) where
read_recorded_only_at_anchor :
∀ sig val anchor sampleT β,
DirectInput sig →
LogicalReadEvent sig val ∈ BucketSigmaEvents C β →
DirectInputAnchorWindow C sig anchor sampleT →
β = AnchorBucket C anchor
```

The late-sample value agreement field is reset-epoch local. It is derived from stableWithinResetEpoch and may be used only when no relevant dynamic reset occurs between the logical anchor cycle and the physical sample cycle.

```
write_recorded_only_at_anchor :
∀ sig val anchor β,
DirectInput sig →
LogicalWriteEvent sig val ∈ BucketSigmaEvents C β →
β = AnchorBucket C anchor

late_sample_is_epsilon_only :
∀ sig anchor sampleT,
DirectInputAnchorWindow C sig anchor sampleT →
```

PhysicalLateSample C sig sampleT →
EpsilonStepOnly C sig sampleT ∧
¬ ∃ val β,
PhysicalSampleEvent sig val sampleT ∈ BucketSigmaEvents C β

late_sample_value_matches_anchor :
∀ sig anchor sampleT,
DirectInputAnchorWindow C sig anchor sampleT →
NoRelevantResetBetween C sig anchor sampleT →
InputValue C.env sig anchor = InputValue C.env sig sampleT

This explicitly reconciles the bucket model with direct-input late sampling:
CompatibleP and CompatibleSys compare the logical Σ-visible Read/Write events
at β_anchor, never the later physical sample micro-step.

---

6. Temporal logic layer for B2/B3/B5/B6

B2 and B3 should not be opaque Prop fields. They are temporal assumptions over infinite executions.

The document defines B2 as weak fairness: if an action remains continuously enabled from some cycle onward, the scheduler must eventually select it. It applies to Push, Pop, and Sync operations. It also defines B3 as the P_push(c) / P_pop(c) channel-progress obligations.

Define temporal operators:

```
def AlwaysFrom
 (τ : InfiniteExecutionBucketed Σ)
 (t : Nat)
 (P : Nat → Prop) : Prop :=
 ∀ n, t ≤ n → P n

def EventuallyFrom
 (τ : InfiniteExecutionBucketed Σ)
 (t : Nat)
 (P : Nat → Prop) : Prop :=
 ∃ n, t ≤ n ∧ P n
```

Concrete fairness actions:

```
inductive FairAction
| push (q : DynMsgInstance)
| pop  (q : DynMsgInstance)
| sync (s : SyncCall)
```

Weak fairness:

```
def WeakFairness
 (C : ComparisonInstance)
 (τ : InfiniteExecutionBucketed C.Sigma_post) : Prop :=
 ∀ a t,
  AlwaysFrom τ t (fun n => FairActionEnabled C τ a n) →
  EventuallyFrom τ t (fun n => FairActionSelected C τ a n)
```

B3 channel-progress predicates should be defined in terms of persistent dynamic request instances, not per-cycle existentially chosen requests.

A blocking request is persistent from cycle t through its commit cycle inclusive when the same dynamic operation instance q keeps its BoundaryReq asserted throughout that interval. For Push operations, persistence also includes payload stability while the request is outstanding, including the commit cycle itself:

```
def PersistentRequestFrom
(C : ComparisonInstance)
(τ : InfiniteExecutionBucketed C.Sigma_post)
(q : DynMsgInstance)
(t : Nat) : Prop :=
Blocking q ∧
∀ n,
t ≤ n →
¬ CommitsBefore C τ q n →
BoundaryReqAt C τ q n ∧
(q.kind = MsgKind.Push →
PayloadAt C τ q n = PayloadAt C τ q t)
```

Equivalently, one may split this into PersistentPushRequestFrom and PersistentPopRequestFrom. The single definition above is preferred here because P_push and P_pop can share the same persistence premise while Push stability is enforced only in the Push case.

```
def BufferedChannel
(C : ComparisonInstance)
(c : Channel) : Prop :=
0 < capacity C c

def RendezvousChannel
(C : ComparisonInstance)
(c : Channel) : Prop :=
capacity C c = 0

def BufferedPushStalledFullBy
(C : ComparisonInstance)
(τ : InfiniteExecutionBucketed C.Sigma_post)
(qPush : DynMsgInstance)
(c : Channel)
(n : Nat) : Prop :=
BufferedChannel C c ∧
qPush.kind = MsgKind.Push ∧
qPush.channel = c ∧
Blocking qPush ∧
BoundaryReqAt C τ qPush n ∧
occAt c n = capacity C c

def BufferedPopStalledEmptyBy
(C : ComparisonInstance)
(τ : InfiniteExecutionBucketed C.Sigma_post)
(qPop : DynMsgInstance)
(c : Channel)
(n : Nat) : Prop :=
BufferedChannel C c ∧
qPop.kind = MsgKind.Pop ∧
qPop.channel = c ∧
Blocking qPop ∧
BoundaryReqAt C τ qPop n ∧
occAt c n = 0

def RendezvousPairRequestingAt
```

```
(C : ComparisonInstance)
(τ : InfiniteExecutionBucketed C.Sigma_post)
(qPush qPop : DynMsgInstance)
(c : Channel)
(n : Nat) : Prop :=
RendezvousChannel C c ∧
MatchingPushRequest qPush c ∧
MatchingPopRequest qPop c ∧
BoundaryReqAt C τ qPush n ∧
BoundaryReqAt C τ qPop n

def RendezvousProgressDischargedBy
(C : ComparisonInstance)
(τ : InfiniteExecutionBucketed C.Sigma_post)
(qPush qPop : DynMsgInstance)
(n : Nat) : Prop :=
MatchingTransferCommitOn C τ qPush qPop n

def MatchingPushRequest
 (qPush : DynMsgInstance)
 (c : Channel) : Prop :=
 qPush.kind = MsgKind.Push ∧
 qPush.channel = c ∧
 Blocking qPush

def MatchingPopRequest
 (qPop : DynMsgInstance)
 (c : Channel) : Prop :=
 qPop.kind = MsgKind.Pop ∧
 qPop.channel = c ∧
 Blocking qPop

def PushProgressDischargedBy
 (C : ComparisonInstance)
 (τ : InfiniteExecutionBucketed C.Sigma_post)
 (qPush qPop : DynMsgInstance)
 (n : Nat) : Prop :=
 CommitAtCycle C τ qPush n ∨
```

```
  CommitAtCycle C τ qPop n ∨
  MatchingTransferCommitOn C τ qPush qPop n

def PopProgressDischargedBy
  (C : ComparisonInstance)
  (τ : InfiniteExecutionBucketed C.Sigma_post)
  (qPop qPush : DynMsgInstance)
  (n : Nat) : Prop :=
  CommitAtCycle C τ qPop n ∨
  CommitAtCycle C τ qPush n ∨
  MatchingTransferCommitOn C τ qPush qPop n
```

Then:

```
def P_push
(C : ComparisonInstance)
(τ : InfiniteExecutionBucketed C.Sigma_post)
(c : Channel) : Prop :=
(BufferedChannel C c →
 ∀ t qPush qPop,
 MatchingPushRequest qPush c →
 MatchingPopRequest qPop c →
 PersistentRequestFrom C τ qPush t →
 PersistentRequestFrom C τ qPop t →
 AlwaysFrom τ t (fun n =>
   BufferedPushStalledFullBy C τ qPush c n) →
 EventuallyFrom τ t (fun n =>
   PushProgressDischargedBy C τ qPush qPop n))
∧
(RendezvousChannel C c →
 ∀ t qPush qPop,
 MatchingPushRequest qPush c →
 MatchingPopRequest qPop c →
 PersistentRequestFrom C τ qPush t →
 PersistentRequestFrom C τ qPop t →
 AlwaysFrom τ t (fun n =>
   RendezvousPairRequestingAt C τ qPush qPop c n) →
```

```
 EventuallyFrom τ t (fun n =>
  RendezvousProgressDischargedBy C τ qPush qPop n))

def P_pop
(C : ComparisonInstance)
(τ : InfiniteExecutionBucketed C.Sigma_post)
(c : Channel) : Prop :=
(BufferedChannel C c →
∀ t qPop qPush,
 MatchingPopRequest qPop c →
 MatchingPushRequest qPush c →
 PersistentRequestFrom C τ qPop t →
 PersistentRequestFrom C τ qPush t →
 AlwaysFrom τ t (fun n =>
 BufferedPopStalledEmptyBy C τ qPop c n) →
 EventuallyFrom τ t (fun n =>
 PopProgressDischargedBy C τ qPop qPush n))
∧
(RendezvousChannel C c →
∀ t qPop qPush,
 MatchingPopRequest qPop c →
 MatchingPushRequest qPush c →
 PersistentRequestFrom C τ qPop t →
 PersistentRequestFrom C τ qPush t →
 AlwaysFrom τ t (fun n =>
 RendezvousPairRequestingAt C τ qPush qPop c n) →
 EventuallyFrom τ t (fun n =>
 RendezvousProgressDischargedBy C τ qPush qPop n))
```

This matches the document's conditional B3 reading: B3 forces progress only when both endpoints continuously request the matching transfer; a producer stalled on a full channel is not automatically a B3 violation if the consumer is not yet requesting the matching pop.

The scheduling document assumes B1 determinism only of the post-HLS RTL_B side. The source-side models Sys / Sys_B / Sys_ref may admit multiple ε- or latency-different executions having the same Σ-projection; witness selection and snooping resolve the required source-side correspondence when needed.

For a fixed comparison instance — including the fixed initial state and fixed reactive environment / external stimulus — B1 requires uniqueness of the labeled post-side Σ-trace, not uniqueness of the full internal ε-execution. Internal ε-steps, ε-depths, and other non-observable scheduling details may differ between valid post-side executions.

def PostDeterministicSigmaTrace
(C : ComparisonInstance) : Prop :=
∀ τ_1 τ_2,
InfinitePostExecution C τ_1 →
InfinitePostExecution C τ_2 →
SigmaTraceOf C .post τ_1 =
SigmaTraceOf C .post τ_2

Bundle B-rules:
structure BRules (C : ComparisonInstance) where
B1_postDeterminism :
PostDeterministicSigmaTrace C
B2_weakFairness :
∀ τ, InfinitePostExecution C τ →
WeakFairness C τ
B3_channelProgress :
∀ τ c, InfinitePostExecution C τ →
P_push C τ c ∧ P_pop C τ c
B4_finiteEpsilonDepth : FiniteEpsilonDepth C
B5_quiescentNormalization : QuiescentNormalization C
B6_idleStutterTimeDivergence : IdleStutterTimeDivergence C

B2/B3 remain assumptions, but now they are structured temporal assumptions usable by K proofs.

---

7. Channel semantics and E4

Define channel state over the selected Sys_ref boundary.

The primitive abstract occupancy is cycle-boundary, non-fall-through occupancy. It is indexed by the cycle, not by the full intra-cycle state:

capacity : C.Sys_ref.Channel → Nat
occAt : C.Sys_ref.Channel → Cycle → Nat

For convenience, one may define a state-indexed abbreviation only by projecting the state to its cycle:

occAtState : C.Sys_ref.Channel → State → Nat
occAtState c σ := occAt c σ.cycle

This abbreviation is not a second occupancy notion. In particular, occupancy is invariant under ε-steps and under same-cycle Σ-events:

occ_invariant_within_cycle :
∀ c σ_1 σ_2,
σ_1.cycle = σ_2.cycle →
occAtState c σ_1 = occAtState c σ_2

def BoundaryReqDomainSide
(C : ComparisonInstance)
(side : Side)
(q : DynMsgInstance)
(σ : State) : Prop :=
∃ P,
q ∈ QOmegaSide C side P ∧
MessageOperationInstance q ∧
OwnsMessageEndpoint P q

Executed failed non-blocking polls may belong to Qexec,P for replay and witness purposes while remaining absent from QOmega,P and Ω_P. Accordingly, they do not acquire BoundaryReq, ChannelEnabled, ChannelDisabled, Front, NextFront, or WFG obligations merely because they executed.

BoundaryReqSide :
ComparisonInstance → Side → DynMsgInstance → State → Prop

ChannelEnabledSide :
ComparisonInstance → Side → DynMsgInstance → State → Prop

```
def ChannelDisabledSide
(C : ComparisonInstance)
(side : Side)
(q : DynMsgInstance)
(σ : State) : Prop :=
BoundaryReqDomainSide C side q σ ∧
BoundaryReqSide C side q σ ∧
¬ ChannelEnabledSide C side q σ
```

For post-side automatic-flush formulas, use the following abbreviations:

```
def BoundaryReqDomain
(C : ComparisonInstance)
(q : DynMsgInstance)
(σ : State) : Prop :=
BoundaryReqDomainSide C .post q σ

def BoundaryReq
(C : ComparisonInstance)
(q : DynMsgInstance)
(σ : State) : Prop :=
BoundaryReqSide C .post q σ

def ChannelEnabled
(C : ComparisonInstance)
(q : DynMsgInstance)
(σ : State) : Prop :=
ChannelEnabledSide C .post q σ

def ChannelDisabled
(C : ComparisonInstance)
(q : DynMsgInstance)
(σ : State) : Prop :=
ChannelDisabledSide C .post q σ
```

BoundaryReqSide, ChannelEnabledSide, and ChannelDisabledSide are the canonical side-indexed predicates. The shorter BoundaryReq / ChannelEnabled /

ChannelDisabled forms are post-side abbreviations used only where the section is already explicitly about RTL_B / post-side automatic-flush behavior.

All of these predicates are used only under BoundaryReqDomainSide or its post-side abbreviation BoundaryReqDomain. Executed failed non-blocking polls in Q_exec,P \ Ω_P are ε-modeled dynamic instances for witness/replay purposes, but they are not message-operation frontier items and do not acquire BoundaryReq, ChannelEnabled, or ChannelDisabled obligations merely because they executed.

ChannelEnabled is allowed to inspect the settled boundary-request state σ, but its capacity/occupancy component must use occAt q.channel σ.cycle. Thus E4 enablement is evaluated against the beginning-of-cycle abstract occupancy, with no same-cycle fall-through through ε-settling or same-cycle Σ-events.

Capacity cases:

```
inductive CapacityKind
| rendezvous   -- B(c) = 0
| buffered     -- B(c) > 0
```

Expected lemmas:

BufferedPushEnabled
BufferedPopEnabled
RendezvousEnabled
FIFOOrderPreservation
NoDropNoDuplicate
OccupancyUpdatePush
OccupancyUpdatePop

E4 should be parameterized by Sys_ref, so in elaborated cases it ranges over explicit elaborated channels, not merely outer channels.

---

8. Appendix Q elaboration interface

Appendix Q should be introduced early. In the updated document, elaboration is not a late proof afterthought: in elaborated cases, Sys_ref = Sys_B^elab(E), and all references to B, occ, BoundaryReq, ChannelEnabled/Disabled, Front_P, and WFG are interpreted over the explicit channels of the elaborated model.

As in the ordinary E4 semantics, elaborated-channel occupancy is cycle-boundary, non-fall-through occupancy. Thus occ_E(d,t) and A_E(c,t) are indexed by cycle t. State-indexed forms such as occ_E(d,σ) or A_E(c,σ), when used informally at ε-quiescent cutpoints, are only abbreviations for occ_E(d,σ.cycle) and A_E(c,σ.cycle).

Chan_outer should be a parameter fixed by the chosen outer Sys_B, not a field chosen internally by the elaboration package.

structure ElaborationPackage
(Sys_B : System)
(Chan_outer : Type)
(Σ_DUT : Type) where

Sys_elab : System

Chan_elab : Type
finite_chan_elab : Fintype Chan_elab

B_E : Chan_elab → Nat
occ_E : Chan_elab → Cycle → Nat

Pi_elab : Chan_elab → Option Chan_outer
piSigma : Trace Sys_elab.Sigma → Trace Σ_DUT
A_E : Chan_outer → Cycle → Nat

outer_trace_preservation : Prop
occupancy_preservation : Prop
internal_refinement : Prop
no_hidden_cross_channel_disablement : Prop
wfg_visibility : Prop
combinational_wellformedness : Prop

For convenience, the elaboration package may also define state-indexed abbreviations by projecting the state to its cycle:

occ_E_atState : Chan_elab → State → Nat
occ_E_atState d σ := occ_E d σ.cycle

A_E_atState : Chan_outer → State → Nat

A_E_atState c σ := A_E c σ.cycle

These abbreviations are not separate occupancy or accounting notions. They are invariant under ε-steps and same-cycle Σ-events:

occ_E_invariant_within_cycle :
∀ d σ_1 σ_2,
σ_1.cycle = σ_2.cycle →
occ_E_atState d σ_1 = occ_E_atState d σ_2

A_E_invariant_within_cycle :
∀ c σ_1 σ_2,
σ_1.cycle = σ_2.cycle →
A_E_atState c σ_1 = A_E_atState c σ_2

Reference choice:

```
inductive ReferenceKind
| ordinarySysB
| elaborated

structure SysRefChoice where
  outerSysB  : System
  Chan_outer : Type
  kind       : ReferenceKind
  elab?      : Option (ElaborationPackage outerSysB Chan_outer Σ_DUT)
  Sys_ref    : System
  sys_ref_matches_kind : Prop
```

For ordinarySysB, Sys_ref = outerSysB.

For elaborated, Sys_ref = E.Sys_elab.

This also handles sync-boundary epoch gates: if proof-relevant withholding exists, it must be represented by explicit gate channels in Sys_B^elab(E), not by hidden gating on an otherwise E4-enabled outer channel. The earlier review correctly flagged this as a point that must be explicit in the Lean strategy.

Locality note. Although ElaborationPackage is presented as a single structure parameterizing the theorem instance, its components (Sys_elab, Chan_elab, Π_elab,

π_Σ,E, A_E) are constructed locally per process and per channel boundary. Each per-process elaboration is determined by that process's local channel structure (number of inputs, sync-boundary discipline, pipelining configuration) and is independent of the elaboration chosen for any other process. The system-level ElaborationPackage is the disjoint composition of these per-process pieces. Treating E as a theorem-instance parameter is the right Lean encoding of "the verifier has chosen per-process elaborations and the proof takes them as given"; it is not a claim that the verifier must reason globally to construct E.

---

9. R-rules, E-rules, and modeling contracts

The R-rule bundle should include only the document's actual R-rules plus an optional Lean-side array-rule pointer if desired. It should not include a fictional R5_syncBoundaryNoCrossing; sync-boundary crossing is E5. The document explicitly labels "Messages cannot cross their immediate synchronization boundary" as E5.

However, R5 itself is a real R-rule: R5 (Channel capacity semantics; Sys vs. Sys_B). In the Lean strategy, R5 should record the chosen source/reference-model capacity convention: raw Sys uses rendezvous capacity, while Sys_B / Sys_ref uses the selected capacity B(c), with B(c)=0 interpreted as rendezvous and B(c)>0 interpreted as cycle-boundary, non-fall-through bounded FIFO. The concrete enablement and occupancy-update obligations remain the E4 channel-capacity semantics; R5 records which source/reference capacity semantics are being used by the selected comparison instance.

```
def R2CCombinationalFixedPointRule
(C : ComparisonInstance) : Prop :=
SequentialOnly C ∨
∃ hR2C : R2C_WF C,
R2CEventsObservedOnlyAtFixedPoint C hR2C

def R5ChannelCapacitySemantics
(C : ComparisonInstance) : Prop :=
SysRefUsesSelectedCapacitySemantics C ∧
∀ c,
(capacity C c = 0 → RendezvousChannel C c) ∧
(0 < capacity C c → BufferedChannel C c)
```

```
structure RRules (C : ComparisonInstance) where
R1_syncOrder : Prop
R2_signalAnchorSequential : Prop
R2C_combinationalFixedPoint :
R2CCombinationalFixedPointRule C
R3_syncSemantics : Prop
R4_messageIssueCommitOrder : Prop
R5_channelCapacitySemantics :
R5ChannelCapacitySemantics C
```

E-rules:

```
structure ERules (C : ComparisonInstance) where
  E1_sync_order_preservation : Prop
  E2_signal_anchor_preservation : Prop
  E3_message_order_preservation : Prop
  E4_channel_capacity_semantics : Prop
  E5_sync_boundary_preservation : Prop
```

Array and direct-input conditions live in:

```
structure ModelingContracts (C : ComparisonInstance) where
  directInputs : DirectInputContract C
  arrays       : ArrayAccessMappingRules C
```

This avoids treating environment/design-side contracts inconsistently.

---

10. Array access mapping

External arrays are not merely internal Store. The document states that external arrays are mapped onto IO operations by an array access mapping layer, and that array accesses before a synchronization call must commit no later than the sync commit, while accesses after the sync must commit strictly after it. It also permits transformations such as caching, merging, splitting, and reordering when allowed by the mapping layer.

Define:

```
inductive MemoryEvent
| read  (array : ArrayId) (addr : Addr) (value : Value)
| write (array : ArrayId) (addr : Addr) (value : Value)
```

```
structure ArrayAccessMapping where
 sourceArrayAccess : SourceArrayAccess → List CoreIOEvent
 commitTime       : SourceArrayAccess → Cycle
 fixedLatency     : SourceArrayAccess → Nat

 conflictFreeReorderAllowed :
  SourceArrayAccess → SourceArrayAccess → Prop

 transformedEquivalent :
  SourceArrayAccess → List SourceArrayAccess → Prop
```

Rules:

```
structure ArrayAccessMappingRules (C : ComparisonInstance) where
 beforeSyncCommitsNoLater :
  ∀ a s,
   a ∈ prefS s →
   ArrayCommitTime a ≤ SyncCommitTime s

 afterSyncCommitsAfter :
  ∀ a s,
   a ∈ suffS s →
   SyncCommitTime s < ArrayCommitTime a

 issueConsistentWithFixedLatency :
  ∀ a,
   ArrayCommitTime a =
   ArrayIssueTime a + ArrayFixedLatency a

 transformedOpsRespectMapping :
  ∀ a transformed,
   TransformedFrom a transformed →
   MappingEquivalent a transformed

 conflictFreeReorderingOnly :
  ∀ a₁ a₂,
   Reordered a₁ a₂ →
   conflictFreeReorderAllowed a₁ a₂
```

For early milestones, it is acceptable to assume:

AllExternalArrayAccessesAlreadyMappedToCoreIO C

but the full proof strategy should include array mapping.

---

11. R2C combinational fixed point

ComparisonInstance now has:

combNetwork? : Option CombNetwork

R2C should use that field when combinational processes are in scope. The document says combinational Read/Write events occur in the observable bucket corresponding to the cycle's ε-quiescent combinational fixed point, and that combinational signal IO is well formed only when the relevant network is deterministic and reaches a unique ε-fixed point.

Define:
structure CombNetwork where
combStep : Store → Store → Prop
deterministic : Prop

The scheduling document's R2C rule requires deterministic ε-settling to a unique observable fixed point. It does not globally require structural combinational acyclicity for every admissible combinational network. Pure combinational oscillation, ambiguous multi-driver resolution, or non-convergent ε-behavior are excluded instead through the unique-fixed-point obligation below.

Structural acyclicity may still be imposed as a separate well-formedness requirement for specific elaboration/coupler constructions in Appendix Q, but it is not a universal requirement of CombNetwork itself.

```
def CombQuiescent (N : CombNetwork) (σ : State) : Prop :=
  ¬ ∃ σ', N.combStep σ.store σ'.store

def CombFixedPointAtCycle
  (N : CombNetwork)
  (t : Cycle)
  (μ : Store) : Prop :=
  ReachesByCombSteps N t μ ∧
  IsFixedPoint N μ
```

Well-formedness:

```
structure R2C_WF (C : ComparisonInstance) where
network_present :
C.combNetwork?.isSome
uniqueFixedPoint :
∀ t N,
C.combNetwork? = some N →
∃! μ, CombFixedPointAtCycle N t μ
sigmaReadsAtFixedPoint :
∀ e t μ N,
C.combNetwork? = some N →
CombReadWriteEvent e →
e ∈ BucketSigmaEvents C t →
CombFixedPointAtCycle N t μ →
EventReadsWritesStore e μ

def R2CEventsObservedOnlyAtFixedPoint
(C : ComparisonInstance)
(hR2C : R2C_WF C) : Prop :=
∀ e t,
CombReadWriteEvent e →
e ∈ BucketSigmaEvents C t →
∃ N μ,
C.combNetwork? = some N ∧
CombFixedPointAtCycle N t μ ∧
EventReadsWritesStore e μ
```

If combinational processes are out of scope for an early milestone, use:

```
SequentialOnly C
```

and postpone R2C_WF.

---

12. Issue/Commit well-formedness

Issue and Commit should be relational plus existence/uniqueness, not necessarily computable total functions.

IssueAt : Trace C.Sys_ref → DynMsgInstance → Cycle → Prop
CommitAt : Trace C.Sys_ref → DynMsgInstance → Cycle → Payload → Prop

Well-formedness:

def BoundaryRequestSoundness
(C : ComparisonInstance) : Prop :=
∀ τ q tIssue t σ,
IssuedInTrace C τ q →
IssueAt τ q tIssue →
StateAtCycle τ t σ →
tIssue ≤ t →
¬ CommitsBefore C τ q t →
Blocking q →
BoundaryReqDomain C q σ ∧
BoundaryReq C q σ ∧
(q.kind = MsgKind.Push →
PayloadAtState C q σ = PayloadAtIssue C τ q tIssue)

def StableAttribution
(C : ComparisonInstance) : Prop :=
∀ τ q_0 q_1 t σt σnext,
SameEndpointDirection q_0 q_1 →
$q_0 \neq q_1$ →
Blocking q_0 →
StateAtCycle τ t σt →
StateAtCycle τ (t + 1) σnext →
BoundaryReqDomain C q_0 σt →
BoundaryReq C q_0 σt →
BoundaryReqDomain C q_1 σnext →
BoundaryReq C q_1 σnext →
¬ EndpointCommitAt C τ q_0.endpoint t →
¬ EndpointCommitAt C τ q_1.endpoint t →
False

This stable-attribution rule applies when the earlier outstanding request q_0 is a blocking Push/Pop. In that case, attribution cannot change to any distinct same-endpoint operation before a commit on that endpoint direction, whether the

later candidate q_1 is blocking or non-blocking. For non-blocking PushNB / PopNB calls, continuous assertion of the endpoint request signal does not by itself collapse later executed non-blocking call instances into the same dynamic instance q; repeated executed NB polls remain distinct source-level dynamic instances unless the witness explicitly identifies the same still-outstanding request instance.

def SameSyncInterval
(C : ComparisonInstance)
(q_0 q_1 : DynMsgInstance) : Prop :=
∀ s,
¬ (q_0 ∈ prefS s ∧ q_1 ∈ suffS s) ∧
¬ (q_1 ∈ prefS s ∧ q_0 ∈ suffS s)

def BackToBackIssueWF
(C : ComparisonInstance) : Prop :=
∀ τ q_0 q_1 t,
SameEndpointDirection q_0 q_1 →
SameSyncInterval C q_0 q_1 →
CommitAtCycle C τ q_0 t →
NextSourceMessageOpOnEndpoint C q_0 q_1 →
EndpointRequestContinuouslyAsserted C τ q_0.endpoint t (t + 1) →
IssueAt τ q_1 (t + 1)

The SameSyncInterval premise prevents the back-to-back issuance rule from carrying a continuously asserted endpoint request across an explicit wait(), SyncChannel, ac_sync, or other synchronization boundary. Cross-boundary issue timing is governed instead by SyncBoundaryIssueDiscipline / E5.

def SyncBoundaryIssueDiscipline
(C : ComparisonInstance) : Prop :=
∀ τ q_0 q_1 s tSync t_0 t_1,
q_0 ∈ prefS s →
q_1 ∈ suffS s →
SyncCommitAt C τ s tSync →
IssueAt τ q_0 t_0 →
IssueAt τ q_1 t_1 →

$t_0 \le tSync \wedge tSync < t_1$

```
def NBDynamicInstanceDiscipline
(C : ComparisonInstance) : Prop :=
∀ P q₀ q₁,
IsNB q₀ →
IsNB q₁ →
ExecutedInProcess C P q₀ →
ExecutedInProcess C P q₁ →
SameSourceCallSiteButDifferentExecution q₀ q₁ →
q₀ ≠ q₁

structure IssueCommitWF (C : ComparisonInstance) where
issue_exists :
∀ τ q,
IssuedInTrace C τ q →
∃ t, IssueAt τ q t
issue_unique :
∀ τ q t₁ t₂,
IssueAt τ q t₁ →
IssueAt τ q t₂ →
t₁ = t₂
commit_exists_unique :
∀ τ q,
CommitsInTrace C τ q →
∃! tp : Cycle × Payload,
CommitAt τ q tp.1 tp.2
boundary_request_soundness :
BoundaryRequestSoundness C
stable_attribution :
StableAttribution C
back_to_back_issue :
BackToBackIssueWF C
sync_boundary_discipline :
SyncBoundaryIssueDiscipline C
nb_dynamic_instance_discipline :
NBDynamicInstanceDiscipline C
```

Derived timestamp:

```
noncomputable def IssueTime
  (hWF : IssueCommitWF C)
  (h : IssuedInTrace C τ q) : Cycle :=
  Classical.choose (hWF.issue_exists τ q h)
```

This preserves Appendix C's use of Issue(q) as an auxiliary timestamp while avoiding an unnecessary computability requirement.

---

13. Automatic flush and sync-boundary policy

Define:
Pending P σ
Front P σ
NextFront P σ
BlockingMsgNextFront P σ
Done P σ
Remain P σ

Automatic flush must be stated over the same dynamic universe and source-order objects used by Appendix I/J/K. In particular, it must not be an opaque Prop bundle.

```
def FrontierBlocked
(C : ComparisonInstance)
(P : Process)
(σ : State) : Prop :=
Front C P σ ≠ ∅ ∧
∀ x,
x ∈ Front C P σ →
BlockingMessageFrontierItem x ∧
BoundaryReq C x σ ∧
ChannelDisabled C x σ

def LaterThanBlockedFrontier
(C : ComparisonInstance)
```

(P : Process)
(σ : State)
(y : FrontierItem) : Prop :=
∃ x,
x ∈ Front C P σ ∧
SourceOrderLt C P x y

def FrontierRemainsBlockedOver
(C : ComparisonInstance)
(P : Process)
(τ : Trace C.RTL_B)
(t_0 t : Cycle) : Prop :=
∀ n σn,
t_0 ≤ n →
n ≤ t →
StateAtCycle τ n σn →
FrontierBlocked C P σn

def NoNewLaterWorkWhileBlocked
(C : ComparisonInstance)
(P : Process)
(τ : Trace C.RTL_B) : Prop :=
∀ t_0 t $σ_0$ y,
StateAtCycle τ t_0 $σ_0$ →
FrontierBlocked C P $σ_0$ →
FrontierRemainsBlockedOver C P τ t_0 t →
LaterThanBlockedFrontier C P $σ_0$ y →
¬ IssueAt τ y t

def NoWithdrawalWhileBlocked
(C : ComparisonInstance)
(P : Process)
(τ : Trace C.RTL_B) : Prop :=
∀ t_0 t $σ_0$ σt x,
StateAtCycle τ t_0 $σ_0$ →
StateAtCycle τ t σt →
t_0 ≤ t →
x ∈ Front C P $σ_0$ →

BoundaryReq C x σ_0 →
¬ CommitsBetween τ x t_0 t →
BoundaryReq C x σt ∧
(IsPushFrontierItem x →
PayloadAtState C x σt = PayloadAtState C x σ_0)

def FrontierMinimality
(C : ComparisonInstance)
(P : Process)
(τ : Trace C.RTL_B) : Prop :=
∀ t σ x y,
StateAtCycle τ t σ →
x ∈ Front C P σ →
y ∈ Remain C P σ →
SourceOrderLt C P y x →
False

The next two predicates are the local automatic-flush instances of the Kε / A11 WFG-inertness obligation. They are stated here because DrainOnlyDownstreamProgress uses them directly; Appendix K later consumes the same predicates as part of the global deadlock-preservation argument.

def WFGInertForProcess
(C : ComparisonInstance)
(P : Process)
(σ_0 σt : State) : Prop :=
∀ Q,
WFGEdge C .post σ_0 P Q ↔ WFGEdge C .post σt P Q

def NoNewBlockingFrontierCause
(C : ComparisonInstance)
(P : Process)
(σ_0 σt : State) : Prop :=
∀ x,
x ∈ Front C .post P σt →
BlockingMessageFrontierItem x →
ChannelDisabledSide C .post x σt →
∃ y,

y ∈ Front C .post P σ_0 ∧
BlockingMessageFrontierItem y ∧
SameBlockingCause C P y x

def DrainOnlyDownstreamProgress
(C : ComparisonInstance)
(P : Process)
(τ : Trace C.RTL_B) : Prop :=
∀ t_0 t σ_0 σt,
StateAtCycle τ t_0 σ_0 →
StateAtCycle τ t σt →
t_0 ≤ t →
FrontierBlocked C P σ_0 →
FrontierRemainsBlockedOver C P τ t_0 t →
InternalOrEpsilonOnlyBetween τ t_0 t →
WFGInertForProcess C P σ_0 σt ∧
NoNewBlockingFrontierCause C P σ_0 σt

structure AutomaticFlush
(C : ComparisonInstance)
(P : Process)
(τ : Trace C.RTL_B) where
block_point :
∀ t σ,
StateAtCycle τ t σ →
FrontierBlocked C P σ →
FlushBlockedPoint C P τ t σ

no_new_later_work :
NoNewLaterWorkWhileBlocked C P τ

no_withdrawal :
NoWithdrawalWhileBlocked C P τ

frontier_minimality :
FrontierMinimality C P τ

drain_only_downstream_progress :

DrainOnlyDownstreamProgress C P τ

sync_boundary_policy :
SyncBoundaryPolicy C P τ

The sync-boundary policy must explicitly split the ordinary and elaborated cases. In the elaborated case, it must also state the content of the withholding rule: producer-facing availability for capacity belonging only to suff_S(s) is unavailable while the synchronization call s is pending.

```
def SyncPendingAt
(C : ComparisonInstance)
(P : Process)
(τ : Trace C.RTL_B)
(s : SyncCall)
(t : Cycle) : Prop :=
SyncIssuedButNotCommitted C P τ s t

def FutureEpochCapacityFor
(C : ComparisonInstance)
(P : Process)
(s : SyncCall)
(gateChannel : Channel)
(q : DynMsgInstance) : Prop :=
q ∈ suffS_msg C P s ∧
MessageOperationInstance q ∧
TransferWouldUseCapacityBehindGate C P s gateChannel q

def SyncBoundaryGateCorrect
(C : ComparisonInstance)
(P : Process)
(τ : Trace C.RTL_B)
(E : ElaborationPackage C.refChoice.outerSysB C.refChoice.Chan_outer C.Sigma_DUT)
(gateChannel : Channel)
(s : SyncCall) : Prop :=
GateChannelOf E gateChannel ∧
∀ t σ q,
StateAtCycle τ t σ →
```

SyncPendingAt C P τ s t →
FutureEpochCapacityFor C P s gateChannel q →
BoundaryReqDomain C q σ →
BoundaryReq C q σ →
ChannelDisabled C q σ

Here suffS_msg C P s is the message-operation-instance projection of suffS(s).
The sync-boundary gate rule is stated over DynMsgInstance values because BoundaryReq, ChannelEnabled, and ChannelDisabled are obligations of attributed message-operation instances, not of arbitrary FrontierItem records.

inductive SyncBoundaryPolicy
(C : ComparisonInstance)
(P : Process)
(τ : Trace C.RTL_B)
| no_withholding_in_ordinary_SysB :
C.refChoice.kind = ReferenceKind.ordinarySysB →
NoProofRelevantSyncBoundaryWithholding C P τ →
SyncBoundaryPolicy C P τ
| explicit_gate_in_elab :
∀ E gateChannel s,
C.refChoice.kind = ReferenceKind.elaborated →
SyncBoundaryGateCorrect C P τ E gateChannel s →
SyncBoundaryPolicy C P τ

In the ordinary Sys_B case, the sync-boundary clause does not add hidden gating. It asserts that no proof-relevant sync-boundary withholding is present. If such withholding is needed, the theorem instance must use Sys_B^elab(E), where the withholding appears as ordinary ChannelDisabled behavior at an explicit sync-boundary / epoch-gate abstract channel.

---

14. Witness selector and Appendix L

The uploaded document's witness selector is a partial strategy over ε-quiescent source cutpoints and ε-modeled choices whose outcomes may affect later Σ-visible behavior; it must be causally available from the matched RTL prefix and global Σ-causal history.

Avoid circularity by defining it over the partial construction so far:

```
structure WitnessSelector (C : ComparisonInstance) where
 choose :
 (τ_post_prefix : Prefix C.RTL_B) →
 (τ_ref_so_far : Prefix C.Sys_ref) →
 (cp : ChoicePoint τ_ref_so_far) →
 Option ChoiceOutcome
 causal :
 ∀ τp τr cp out,
 choose τp τr cp = some out →
 DependsOnlyOnAvailablePrefix τp τr cp out
 consistent_with_post_prefix :
 ∀ τp τr cp out,
 choose τp τr cp = some out →
 ConsistentWithRTLPrefix τp τr cp out
```

Non-blocking witness mapping.
Failed PushNB / PopNB polls are executed dynamic instances in Q_exec but produce no Σ-event. Therefore the Post → Ref construction cannot reconstruct them from the Σ-trace alone. The witness bundle must carry an explicit partial dynamic-instance correspondence for such ε-modeled NB instances.

```
structure NBInstanceMap (C : ComparisonInstance) where
μ_Q :
DynMsgInstance → Option DynMsgInstance

maps_only_executed_nb :
∀ q_post q_ref,
μ_Q q_post = some q_ref →
ExecutedInPost C q_post ∧
ExecutedInRef C q_ref ∧
IsNB q_post ∧ IsNB q_ref

preserves_owner_and_interface :
∀ q_post q_ref,
μ_Q q_post = some q_ref →
q_post.proc = q_ref.proc ∧
q_post.channel = q_ref.channel ∧
q_post.kind = q_ref.kind
```

preserves_nb_outcome :
∀ q_post q_ref,
μ_Q q_post = some q_ref →
NBOutcomePost C q_post = NBOutcomeRef C q_ref

covers_control_relevant_failed_polls :
∀ q_post,
ExecutedInPost C q_post →
IsFailedNB q_post →
AffectsLaterSigmaOrFrontier C q_post →
∃ q_ref, μ_Q q_post = some q_ref

source_order_position_preserved :
∀ q_post q_ref,
μ_Q q_post = some q_ref →
SourceOrderKeyOf C q_post = SourceOrderKeyOf C q_ref

respects_source_order_slice :
∀ P q_1_post q_2_post q_1_ref q_2_ref,
μ_Q q_1_post = some q_1_ref →
μ_Q q_2_post = some q_2_ref →
SourceOrderLe C P (SourceOrderKeyOf C q_1_post) (SourceOrderKeyOf C q_2_post) ↔
SourceOrderLe C P (SourceOrderKeyOf C q_1_ref) (SourceOrderKeyOf C q_2_ref)

structure WitnessBundle (C : ComparisonInstance) where
selector : WitnessSelector C
nbMap : NBInstanceMap C

Witness compatibility:
def WC
(C : ComparisonInstance)
(W : WitnessBundle C) : Prop :=
(∀ constructionPrefix,
WitnessChoicesRespectMatchedPrefix C W.selector constructionPrefix) ∧
WitnessNBMapRespectsMatchedPrefix C W.nbMap ∧
WitnessChoicesAndNBMapAgree C W.selector W.nbMap

Thus WC is the Lean-side packaging of both witness selection and the μ_Q matching obligation. In particular, when E3/E5 reasoning or the Post → Ref construction refers to failed ε-modeled non-blocking call instances in Q_exec,P, the required μ_Q correspondence is supplied by W.nbMap as part of WC, not as a separate unrelated assumption.

Lemma L.2:
```
theorem L2_snooping_establishes_WC
 (C : ComparisonInstance)
 (W : WitnessBundle C)
 (hSnoop : SnoopingWrapperAssumptions C W) :
 WC C W
```

IdentityOrArbitraryNBWitnessBundle C is the witness bundle used in the NB-CF case. Its selector uses identity choices for ε-modeled non-blocking outcomes that are fixed by the matched prefix, and arbitrary choices for non-blocking outcomes that NB-CF proves cannot affect later Σ-visible control flow or NextFront behavior. Its nbMap maps matched executed NB instances to the corresponding same dynamic source-call instance when such a match is relevant, and is arbitrary on failed NB instances that NB-CF proves irrelevant.

```
def IdentityOrArbitraryNBWitnessBundle
(C : ComparisonInstance) : WitnessBundle C :=
{
selector := IdentityOrArbitraryNBSelector C,
nbMap := IdentityOrArbitraryNBMap C
}
```

Corollary L.3:
```
theorem L3_nb_cf_identity_witness
(C : ComparisonInstance)
(P : Process)
(hNBCF : NBCF C P) :
WC C (IdentityOrArbitraryNBWitnessBundle C)
```

---

15. Concrete ObsSigma and deterministic replay

Do not define ObsSigma as an informal least fixed point. Define it as a concrete record of the source-state slices required for replay.

structure ObsSigma
(C : ComparisonInstance)
(W : WitnessBundle C)
(P : Process)
(σ : State) where
control_pos : ControlPos P
relevant_vars : RelevantVarSlice C P σ
selected_eps_outcomes : SelectedOutcomeSlice C W.selector P σ
nb_witness_slice : NBWitnessSlice C W.nbMap P σ
msg_attribution_slice : MsgAttributionSlice C W P σ
pending_slice : PendingSlice C P σ
front_slice : FrontSlice C P σ
nextfront_slice : NextFrontSlice C P σ
boundary_req_slice : BoundaryReqSlice C P σ
channel_fact_slice : ChannelFactSlice C P σ
signal_anchor_slice : SignalAnchorSlice C P σ
array_mapping_slice : ArrayMappingSlice C P σ
direct_input_slice : DirectInputSlice C P σ

Here msg_attribution_slice records the Q_exec,P / Q_Ω,P attribution facts needed for E3/E5, Issue/Commit replay, WFG construction, and BoundaryReq ownership. It is broader than nb_witness_slice: nb_witness_slice records μ_Q matching for ε-modeled failed NB calls, while msg_attribution_slice records the source-level operation attribution of pending and frontier message-operation instances generally.

Then:
def SigmaRelevantEq C W P σ_1 σ_2 :=
ObsSigma C W P σ_1 = ObsSigma C W P σ_2

Replay theorem:
theorem IDR_deterministic_replay
(C : ComparisonInstance)
(W : WitnessBundle C)
(hB : BRules C)
(hR : RRules C)

(hE : ERules C)
(hIC : IssueCommitWF C)
(hContracts : ModelingContracts C)
(hWC : WC C W)
(P : Process)
(σ_1 σ_2 : State)
(hObs : SigmaRelevantEq C W P σ_1 σ_2) :
SameNextSigmaRelevantBehavior C W P σ_1 σ_2

This same ObsSigma object should be reused by Lemma S1 and Corollary J.1.

---

16. Appendix I per-process construction

Bundle the assumptions:

structure IConstructionAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C) where
wellformed : C.WellFormed
bRules : BRules C
rRules : RRules C
eRules : ERules C
issueCommitWF : IssueCommitWF C
implementation : ImplementationAssumptions C
witnessCompatible : WC C W
contracts : ModelingContracts C

Main theorems:
theorem I1_bounded_epsilon_closure
(C : ComparisonInstance)
(hB4 : C.assumptions.B.B4_finiteEpsilonDepth)
(hB5 : C.assumptions.B.B5_quiescentNormalization) :
...

theorem I2_channel_progress_instance
(C : ComparisonInstance)
(hB3 : C.assumptions.B.B3_channelProgress) :
...

theorem I3_finite_extension_step
(C : ComparisonInstance)
(W : WitnessBundle C)
(hI : IConstructionAssumptions C W) :
...

theorem I4_finite_prefix_construction
(C : ComparisonInstance)
(W : WitnessBundle C)
(hI : IConstructionAssumptions C W) :
...

theorem I5_infinite_limit_construction
(C : ComparisonInstance)
(W : WitnessBundle C)
(hI : IConstructionAssumptions C W) :
...

I3 should use IssueAt plus derived IssueTime, not an unconstrained guessed issue cycle.

---

17. Appendix J system-level correspondence

Define execution scopes:

def Exec_post (C : ComparisonInstance) (τ : Prefix C.RTL_B) : Prop :=
∃ τ∞,
Extends τ∞ τ ∧
InfinitePostExecution C τ∞ ∧
FairProgressSatisfying C τ∞

def Exec_ref (C : ComparisonInstance) (τ : Prefix C.Sys_ref) : Prop :=
∃ τ∞,
Extends τ∞ τ ∧
RefAdmissible C τ∞

Here FairProgressSatisfying includes the B2/B3 fairness/progress assumptions

on infinite executions. Deadlock cutpoints are admitted into Exec_post by the B6 idle-stutter convention: once a reachable ε-quiescent deadlock cutpoint is reached, it may be extended by infinite idle stuttering so that the prefix has an infinite extension. The theorem deadlock_cutpoint_in_Exec_post below records that connection explicitly.

Directed compatibility:
Do not define CompatibleP or CompatibleSys as equality of bucketed executions.

First define the observable units extracted from a bucketed execution. An observable unit may be:
• a single ordinary Σ-event;
• an explicitly atomic multi-endpoint unit, such as a rendezvous Push/Pop pair or a SyncChannel/ac_sync pair;
• a same-cycle finite set of independent Σ-events equipped with the partial order actually mandated by E1–E5.

structure ObservableUnit (Σ : Type) where
events : Finset Σ
order : PartialOrderOn events
atomic : Prop

ObservableUnitsOf extracts Σ-visible events from the bucketed execution and groups them into comparison units as follows:
• an ordinary committed message, signal, or synchronization event may form a singleton observable unit;
• explicitly atomic multi-endpoint events, such as rendezvous Push/Pop pairs and SyncChannel/ac_sync handshakes, form one atomic observable unit and may not be split by CompatibleP or CompatibleSys;
• R2C combinational observations are taken from the ε-quiescent fixed-point bucket and grouped according to the R2C well-formedness relation;
• same-cycle independent events may remain unordered within a unit only when no E1–E5, channel, synchronization, or witness-attribution rule orders them;
• same-cycle events that are merely coincident in one implementation are not forced to remain same-cycle in the other implementation unless they are explicitly atomic or otherwise ordered.

def ObservableUnitsOf

(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side)) :
Set (ObservableUnit (SigmaOfSide C side)) :=
ExtractSigmaUnits
C side τ
(AtomicRendezvousUnits C side τ)
(AtomicSyncUnits C side τ)
(R2CFixedPointUnits C side τ)
(IndependentSameCycleUnits C side τ)

HappensBefore is the transitive closure of the order generators that the document requires CompatibleP / CompatibleSys to preserve:

- per-process E1 synchronization order;
- E2 signal-anchor order;
- E3 same-interface order and distinct-interface no-reverse order;
- E4 per-channel FIFO/rendezvous order and capacity-derived dependency order;
- E5 synchronization-boundary preservation;
- explicit atomic-unit membership constraints;
- same-cycle partial order recorded inside a CycleBucket when it reflects a real required dependency rather than an arbitrary linearization;
- witness-selected NB/control dependencies that affect later Σ-visible behavior, NextFront, or boundary-request attribution;
- elaboration-projection constraints when Sys_ref = Sys_B^elab(E).

def HappensBefore
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side))
(u v : ObservableUnit (SigmaOfSide C side)) : Prop :=
Relation.TransGen
(HappensBeforeGenerator C side τ)
u v

def HappensBeforeGenerator
(C : ComparisonInstance)
(side : Side)

(τ : Prefix (SystemOfSide C side))
(u v : ObservableUnit (SigmaOfSide C side)) : Prop :=
E1SyncOrder C side τ u v ∨
E2SignalAnchorOrder C side τ u v ∨
E3MessageOrder C side τ u v ∨
E4ChannelOrder C side τ u v ∨
E5SyncBoundaryOrder C side τ u v ∨
AtomicUnitOrder C side τ u v ∨
RequiredSameCycleBucketOrder C side τ u v ∨
WitnessControlOrder C side τ u v ∨
ElaborationProjectionOrder C side τ u v

CompatibleP : ComparisonInstance → Process → Prefix C.Sys_ref → Prefix C.RTL_B → Prop

CompatibleP C P τ_ref τ_post means:
• the P-local observable units of τ_ref and τ_post are related by a value-label-preserving bijection;
• explicitly atomic units are mapped to explicitly atomic units and are not split;
• the mapping preserves the P-local happens-before constraints induced by E1, E2, E3, E5, R2/R2C, and the selected witness/request-attribution facts;
• it does not require independent P-local units to occupy the same cycle bucket in both executions.

CompatibleSys : ComparisonInstance → Prefix C.Sys_ref → Prefix C.RTL_B → Prop

CompatibleSys C τ_ref τ_post means:
• ∀ P, CompatibleP C P τ_ref τ_post;
• all explicit abstract channels in the chosen Sys_ref / RTL_B comparison satisfy the applicable E4 channel semantics at the agreed abstract boundaries;
• the system-level observable-unit mapping preserves the mandated global happens-before / ≺_J constraints and value labels;
• explicitly atomic multi-endpoint units remain atomic on both sides;
• no equality of CycleBucket lists, no equality of bucket cycle numbers, and no bucket-by-bucket equality of sigmaEvents is required.

Thus the bijection used by CompatibleSys is over ObservableUnitsOf, and its

ordering obligation is preservation of HappensBefore. Independent observable units may be placed in different cycle buckets or grouped differently in Sys_ref and RTL_B, provided their Σ labels, values, atomicity requirements, and required happens-before constraints are preserved.

The notation τ_ref,system ≈_sys τ_post,system is an abbreviation for CompatibleSys C τ_ref τ_post. It is a directed compatibility predicate, not a symmetric equivalence relation. The main construction direction is Post → Ref; the reverse direction is restricted to RTL-consistent source prefixes.

J.0:

```
theorem J0_pairwise_composition
 (C : ComparisonInstance)
 (hC : C.WellFormed)
 (hB : BRules C)
 (hR : RRules C)
 (hE : ERules C)
 (hIC : IssueCommitWF C)
 (hContracts : ModelingContracts C)
 (τ_ref : Prefix C.Sys_ref)
 (τ_post : Prefix C.RTL_B)
 (hProc : ∀ P, CompatibleP C P τ_ref τ_post)
 (hChan : ChannelsWellFormedAndMatched C τ_ref τ_post) :
 CompatibleSys C τ_ref τ_post
```

J.1:

```
structure JAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C) where
wellformed : C.WellFormed
bRules : BRules C
rRules : RRules C
eRules : ERules C
issueCommitWF : IssueCommitWF C
implementation : ImplementationAssumptions C
witnessCompatible : WC C W
contracts : ModelingContracts C
```

environment : ReactiveEnvironmentWF C.env

theorem J1_post_to_ref
(C : ComparisonInstance)
(W : WitnessBundle C)
(hJ : JAssumptions C W)
(τ_post : Prefix C.RTL_B)
(hExec : Exec_post C τ_post) :
∃ τ_ref,
Exec_ref C τ_ref ∧
CompatibleSys C τ_ref τ_post

Restricted reverse:
theorem J1_ref_to_post_restricted
(C : ComparisonInstance)
(W : WitnessBundle C)
(hJ : JAssumptions C W)
(τ_ref : Prefix C.Sys_ref)
(hExec : Exec_ref C τ_ref)
(hConsistent : RTLConsistentSourcePrefix C W τ_ref) :
∃ τ_post,
Exec_post C τ_post ∧
CompatibleSys C τ_ref τ_post

Cutpoint correspondence:

structure CutpointCorrespondence
 (C : ComparisonInstance)
 (σ_ref : State)
 (σ_post : State) where

 nextfront_agreement : Prop
 frontier_guard_value_boundaryreq_agreement : Prop
 buffered_channel_state_agreement : Prop
 raw_rendezvous_endpoint_request_agreement : Prop
 pending_blocking_enablement_agreement : Prop

direct_input_anchor_agreement : Prop
array_mapping_agreement : Prop
r2c_fixed_point_agreement : Prop

theorem J1_cutpoint_correspondence
(C : ComparisonInstance)
(W : WitnessBundle C)
(hJ : JAssumptions C W)
(τ_ref : Prefix C.Sys_ref)
(τ_post : Prefix C.RTL_B)
(hCompat : CompatibleSys C τ_ref τ_post)
(hNorm : EpsilonNormalizedPair C τ_ref τ_post) :
∃ σ_ref σ_post,
TerminalCutpoint τ_ref σ_ref ∧
TerminalCutpoint τ_post σ_post ∧
CutpointCorrespondence C σ_ref σ_post

---

18. Appendix K WFG and liveness

Define WFG over selected Sys_ref boundaries:
def WFG
(C : ComparisonInstance)
(side : Side)
(σ : State) : Digraph Process :=
{ edge := WFGEdge C side σ }

def WFGEdge
(C : ComparisonInstance)
(side : Side)
(σ : State)
(P Q : Process) : Prop :=
∃ q,
q ∈ Front C side P σ ∧
BlockingMessageFrontierItem q ∧
BoundaryReqSide C side q σ ∧
ChannelDisabledSide C side q σ ∧
ComplementOwner C side q = Q ∧
InternalChannel C side q.channel

Edges are generated from Front_P(σ), not arbitrary non-frontier asserted requests. This intentionally makes the WFG sparser than a graph over all asserted boundary requests: only pending blocking frontier items generate wait-for edges. Failed non-blocking polls and already-asserted non-frontier requests may affect state or occupancy through other rules, but they do not themselves create WFG dependency edges.

```
def ProcessStalledAtFront
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State) : Prop :=
Front C side P σ ≠ ∅ ∧
∀ x,
x ∈ Front C side P σ →
BlockingMessageFrontierItem x →
BoundaryReqSide C side x σ →
ChannelDisabledSide C side x σ

def WFGClosed
(C : ComparisonInstance)
(side : Side)
(D : Finset Process)
(σ : State) : Prop :=
∀ P Q,
P ∈ D →
WFGEdge C side σ P Q →
Q ∈ D

def WFGTerminalSCC
(C : ComparisonInstance)
(side : Side)
(D : Finset Process)
(σ : State) : Prop :=
D.Nonempty ∧
StronglyConnectedSubgraph (WFG C side σ) D ∧
WFGClosed C side D σ
```

```
def DrainClosed
(C : ComparisonInstance)
(side : Side)
(D : Finset Process)
(σ : State) : Prop :=
∀ P,
P ∈ D →
NoEnabledDrainStepLeavingD C side P D σ

def ExistsClosedDrainClosedWFGCycle
(C : ComparisonInstance)
(side : Side)
(σ : State) : Prop :=
∃ D,
WFGTerminalSCC C side D σ ∧
DrainClosed C side D σ ∧
∀ P,
P ∈ D →
ProcessStalledAtFront C side P σ

def InternalMPDeadlockAt
(C : ComparisonInstance)
(side : Side)
(σ : State) : Prop :=
EpsQuiescent σ ∧
ExistsClosedDrainClosedWFGCycle C side σ
```

The K.1 SCC contradiction should use this definition. The closed SCC supplies the cycle of internal message-passing dependencies; DrainClosed rules out escape by downstream draining; ProcessStalledAtFront ensures every process in D is stalled on its current blocking frontier, not merely carrying a non-frontier asserted request.

K assumptions should preserve the document's A1–A11 as a bundle, and add Lean-side auxiliary contracts without pretending they are doc-numbered assumptions.

```
def A11_Kepsilon_WFGInert
(C : ComparisonInstance) : Prop :=
```

∀ P τ t_0 t σ_0 σt,
StateAtCycle τ t_0 σ_0 →
StateAtCycle τ t σt →
$t_0 \le t$ →
InternalOrEpsilonOnlyBetween τ t_0 t →
FrontierBlocked C P σ_0 →
FrontierRemainsBlockedOver C P τ t_0 t →
WFGInertForProcess C P σ_0 σt ∧
NoNewBlockingFrontierCause C P σ_0 σt

structure DocKAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C) where
A1_point_to_point_channels : Prop
A2_chosen_Sys_ref_boundaries : Prop
A3_R_E_rule_conformance : RRules C ∧ ERules C
A4_B_rules : BRules C
A5_source_liveness : InternalMPDeadlockFree C.Sys_ref
A6_issue_commit_wf : IssueCommitWF C
A7_witness_bundle : WC C W
A8_epsilon_normalization : Prop
A9_single_transfer_per_cycle : Prop
A10_reset_empty_or_initial_accounting : Prop
A11_Kepsilon_WFG_inert :
A11_Kepsilon_WFGInert C

def KElaborationPrerequisite
(C : ComparisonInstance) : Prop :=
∀ c,
ChannelCanContributeWFGEdge C c →
ChosenAbstractBoundaryOfSysRef C c ∧
NoHiddenSyncBoundaryGatingAtOrdinaryBufferedBoundary C c ∧
(RequiresSyncBoundaryWithholding C c →
C.refChoice.kind = ReferenceKind.elaborated)

Lean-side additions:
structure KAssumptions
(C : ComparisonInstance)

(W : WitnessBundle C) where
doc : DocKAssumptions C W
contracts : ModelingContracts C
r2c? : Option (R2C_WF C)
elaboration_prerequisite :
KElaborationPrerequisite C

This avoids fictional A12/A13 labels.

K lemmas:
theorem K0_frontier_blocking_classification
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W)
(σ : State)
(hQ : EpsQuiescent σ) :
...

theorem K0a_rendezvous_endpoint_request_agreement
(C : ComparisonInstance)
(W : WitnessBundle C)
(hCorr : CutpointCorrespondence C σ_ref σ_post)
(c : Channel)
(hB0 : capacity c = 0) :
Req_prod c σ_ref = Req_prod c σ_post ∧
Req_cons c σ_ref = Req_cons c σ_post

theorem K1_deadlock_characterization
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W)
(σ : State)
(hReach : Reachable C.RTL_B σ)
(hQ : EpsQuiescent σ) :
InternalMPDeadlockAt C .post σ ↔
ExistsClosedDrainClosedWFGCycle C .post σ

This theorem uses hK.elaboration_prerequisite: every channel boundary used by the WFG is the chosen abstract boundary of the Sys_ref / RTL_B comparison after any required elaboration. Synchronization-boundary ramp-down / epoch-gating is therefore represented at explicit Sys_B^elab channels, not as a hidden exception to E4 on an otherwise ordinary buffered channel.

theorem K2_dependency_refinement
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W)
(σ_ref σ_post : State)
(hCorr : CutpointCorrespondence C σ_ref σ_post) :
WFG C .ref σ_ref = WFG C .post σ_post

theorem K2a_drain_closure_transfer
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W)
(D : Finset Process)
(σ_ref σ_post : State)
(hCorr : CutpointCorrespondence C σ_ref σ_post)
(hDrainPost : DrainClosed C .post D σ_post) :
DrainClosed C .ref D σ_ref

Exec-post membership at a deadlock cutpoint should be explicit:

theorem deadlock_cutpoint_in_Exec_post
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W)
(σ_post : State)
(hReach : Reachable C.RTL_B σ_post)
(hDead : ExistsClosedDrainClosedWFGCycle C .post σ_post)
(hIdle : C.assumptions.B.B6_idleStutterTimeDivergence) :
∃ τ_post,
TerminalCutpoint τ_post σ_post ∧
Exec_post C τ_post

Main K theorem:
theorem K1_liveness_preservation
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W) :
InternalMPDeadlockFree C.Sys_ref →
InternalMPDeadlockFree C.RTL_B

This is intentionally one-way. Do not state:

InternalMPDeadlockFree C.Sys_ref ↔ InternalMPDeadlockFree C.RTL_B

---

19. Appendix R facade

Appendix R should be formalized last as aliases/wrappers over earlier theorems.

Use Lean names that avoid the doc's R.2 collision:

theorem R_lemma1_front_nextfront_classification := ...
theorem R_theorem1_prefix_matching := ...
theorem R_corollary1_cutpoint_correspondence := ...
theorem R_corollary1a_rendezvous_endpoint_request_agreement := ...
theorem R_lemma2_dependency_refinement := K2_dependency_refinement
theorem R_theorem2_closed_cycle_and_one_way_drain_transfer := ...

Do not call cutpoint correspondence R1_cutpoint_correspondence; in the doc structure, it corresponds to a corollary, not Theorem R.1.

---

20. What remains as hypotheses

These remain theorem parameters or design/tool obligations:

1. R-rules R1, R2, R2C, R3, R4.
2. E-rules E1–E5.
3. B1–B6, with B2/B3 encoded temporally.
4. Implementation assumptions such as I.A0.
5. Witness compatibility WC, derived from Lemma L.2 or Corollary L.3 where applicable.

6. Direct-input contracts.
7. Array access mapping rules.
8. Elaboration package E, when Sys_ref = Sys_B^elab(E).
9. Source-level internal message-passing deadlock freedom for K.

---

21. What should be proved inside Lean

Lean should prove:

- bucketed execution infrastructure;
- temporal lemmas for AlwaysFrom / EventuallyFrom;
- channel-capacity lemmas for rendezvous and buffered cases;
- issue existence/uniqueness and derived IssueTime;
- direct-input late-sampling preservation from primitive environment contracts;
- array sync-boundary commit-order preservation;
- R2C fixed-point observation lemmas from R2C_WF;
- automatic-flush frontier lemmas;
- Lemma L.2: snooping establishes WC;
- Corollary L.3: NB-CF identity witness;
- I.DR over concrete ObsSigma;
- Appendix I finite and infinite witness construction;
- Proposition J.0;
- Theorem J.1 post-to-reference;
- restricted reverse theorem for RTL-consistent source prefixes;
- Corollary J.1 cutpoint correspondence;
- K.0/K.0a/K.1/K.2/K.2a;
- deadlock cutpoint membership in Exec_post;
- Theorem K.1 one-way liveness preservation;

- Appendix R aliases.

---

22. Development milestones

Milestone A — Sequential, blocking-only, non-elaborated core

Scope:

- Sys_ref = Sys_B;
- sequential processes only;
- no NB calls;
- no snooping;
- no external arrays;
- no direct-input late sampling;
- no combinational R2C;
- bucketed execution model.

Targets:

J1_post_to_ref_blocking
J1_cutpoint_correspondence_blocking
K1_liveness_preservation_blocking

Milestone B — Temporal B2/B3

Add:

- infinite bucketed executions;
- AlwaysFrom, EventuallyFrom;
- concrete FairAction;
- P_push, P_pop.

Target:

K1_liveness_preservation_with_temporal_progress

Milestone C — Non-blocking under NB-CF

Add:

- Qexec;
- failed NB polls as ε;
- successful NB transfers as Σ;
- Corollary L.3.

Target:

J1_post_to_ref_nb_cf

Milestone D — Witness selector / snooping

Add:

- partial-prefix WitnessSelector;
- Lemma L.2.

Target:

J1_post_to_ref_with_snooping

Milestone E — Direct inputs

Add:

- DirectInputContract;
- derived lateSamplingPreservesLogicalAnchor;
- bucket placement for logical anchor vs physical late sample.

Target:

J1_post_to_ref_direct_inputs

Milestone F — External arrays

Add:

- MemoryEvent;
- ArrayAccessMapping;
- sync-relative array commit rules.

Target:

J1_post_to_ref_external_arrays

Milestone G — R2C

Add:

- combNetwork?;
- CombNetwork;
- R2C_WF;
- fixed-point bucket observation semantics.

Target:

J1_post_to_ref_with_R2C

Milestone H — Elaboration

Add:

- ElaborationPackage Sys_B Chan_outer Σ_DUT;
- Π_elab;
- π_Σ,E;
- A_E;
- explicit-channel interpretation of B, occ, BoundaryReq, ChannelEnabled, Front, and WFG.

Targets:

J1_post_to_ref_elab
K1_liveness_preservation_elab

Milestone I — Appendix R facade

Add compact R theorem wrappers after the full proof stack is available.

---

23. Expected difficulty

The hardest pieces remain:

1. Bucketed execution and ideal induction.
2. Temporal B2/B3 over infinite executions.
3. Concrete ObsSigma and I.DR.

4. Appendix J finite-ideal construction.
5. Direct-input late-sampling contracts.
6. External array access mapping.
7. R2C fixed-point semantics.
8. Appendix Q elaboration obligations.
9. Appendix K Exec_post membership at deadlock cutpoints.

A polished full formalization is plausibly in the 18k–30k Lean line range if direct inputs, arrays, R2C, temporal fairness/progress, non-blocking witness selection, snooping, automatic flush, Issue/Commit attribution, A11/Kε, and elaboration are all included. A core sequential/blocking/non-elaborated proof would be substantially smaller, plausibly 6k–10k lines; an intermediate blocking-plus-temporal-WFG proof would likely fall around 10k–16k lines.